

Guia SQL de Basico a Experto

Coleccion original de teoria, ejercicios y solucionario usando la base Sakila

Que incluye

- Ruta progresiva para estudiar SQL
- 11 modulos desde SELECT hasta funciones de ventana y analisis reales
- 110 ejercicios originales con soluciones
- Enfoque en agregaciones, castings, fechas, joins, CTEs y casos de negocio
- Pensado para practicar en Jupyter con Sakila

Nota: esta guia es original. Esta inspirada en la progresion tipica de estudio de SQL, pero no copia ejercicios de libros comerciales.

Tabla	Uso principal
actor	Catalogo de actores
film	Catalogo principal de peliculas
customer	Clientes
payment	Pagos realizados
rental	Eventos de renta
inventory	Inventario por tienda
category / film_category	Clasificacion por categoria
film_actor	Relacion pelicula-actor
address / city / country	Ubicacion geografica

Recomendacion de estudio: recorre un modulo por vez, ejecuta cada ejemplo, intenta resolver los ejercicios sin ver la respuesta y luego compara tu solucion con el solucionario.

0. Puesta en marcha en Jupyter con ipython-sql

Si vas a trabajar en notebooks, una forma comoda es usar **ipython-sql** con **PyMySQL**.

```
%load_ext sql
from urllib.parse import quote_plus

user = 'root'
password = quote_plus('TU_PASSWORD')
host = 'localhost'
port = '3306'
db = 'sakila'

conn = f"mysql+pymysql://{user}:{password}@{host}:{port}/{db}"
%sql $conn
```

Luego puedes ejecutar consultas con celdas **%%sql**.

```
%%sql
SELECT *
FROM actor
LIMIT 5;
```

Plan sugerido de estudio

- Semana 1: Modulos 1 a 3. Domina lectura, filtros, alias, CASE y funciones basicas.
- Semana 2: Modulos 4 y 5. Enfocate en agregaciones y joins; aqui empieza el SQL de analista de datos.
- Semana 3: Modulos 6 a 8. Practica fechas, castings, subconsultas y CTEs.
- Semana 4: Modulos 9 a 11. Ventanas, casos reales, calidad y validacion.

Modulo 1. SELECT, alias, ORDER BY y LIMIT

Objetivo: Aprender a leer tablas, proyectar columnas, renombrar salidas y ordenar resultados.

Conceptos clave

- SELECT elige columnas; * sirve para explorar, pero no es buena practica en reportes finales.
- AS crea alias legibles para negocio y para notebooks.
- ORDER BY define el orden final del resultado; puede ordenar por varias columnas.
- LIMIT ayuda a revisar muestras rapidamente mientras desarrollas una consulta.

Ejemplos guiados

Muestra 5 actores

```
SELECT *  
FROM actor  
LIMIT 5;
```

Lista titulos y duracion

```
SELECT title AS titulo, length AS duracion_min  
FROM film  
ORDER BY duracion_min DESC  
LIMIT 10;
```

Ejercicios

- 1.1 Muestra todas las columnas de actor y limita a 10 filas.
- 1.2 Lista first_name y last_name de actor.
- 1.3 Muestra title, release_year y rating de film.
- 1.4 Ordena las peliculas por title ascendente.
- 1.5 Ordena las peliculas por length descendente y luego por title ascendente.
- 1.6 Muestra las 15 peliculas mas largas.
- 1.7 Lista clientes con alias nombre_completo concatenando nombre y apellido.
- 1.8 Muestra customer_id, email y active con alias en espanol.
- 1.9 Lista las ciudades alfabeticamente y limita a 20.
- 1.10 Muestra las 10 primeras direcciones ordenadas por district y address.

Solucionario

1.1 Muestra todas las columnas de actor y limita a 10 filas.

```
SELECT *  
FROM actor  
LIMIT 10;
```

1.2 Lista first_name y last_name de actor.

```
SELECT first_name, last_name  
FROM actor;
```

1.3 Muestra title, release_year y rating de film.

```
SELECT title, release_year, rating
FROM film;
```

1.4 Ordena las peliculas por title ascendente.

```
SELECT title
FROM film
ORDER BY title ASC;
```

1.5 Ordena las peliculas por length descendente y luego por title ascendente.

```
SELECT title, length
FROM film
ORDER BY length DESC, title ASC;
```

1.6 Muestra las 15 peliculas mas largas.

```
SELECT title, length
FROM film
ORDER BY length DESC
LIMIT 15;
```

1.7 Lista clientes con alias nombre_completo concatenando nombre y apellido.

```
SELECT CONCAT(first_name, ' ', last_name) AS nombre_completo
FROM customer;
```

1.8 Muestra customer_id, email y active con alias en espanol.

```
SELECT customer_id AS id_cliente, email AS correo, active AS activo
FROM customer;
```

1.9 Lista las ciudades alfabeticamente y limita a 20.

```
SELECT city
FROM city
ORDER BY city
LIMIT 20;
```

1.10 Muestra las 10 primeras direcciones ordenadas por district y address.

```
SELECT district, address
FROM address
ORDER BY district, address
LIMIT 10;
```

Modulo 2. WHERE, operadores logicos, IN, BETWEEN, LIKE y NULL

Objetivo: Filtrar informacion con condiciones simples y compuestas.

Conceptos clave

- WHERE filtra filas antes del ORDER BY.
- IN simplifica multiples comparaciones de igualdad.
- BETWEEN incluye ambos extremos del rango.
- LIKE usa % para varios caracteres y _ para uno.
- IS NULL e IS NOT NULL se usan con valores nulos; = NULL no funciona.

Ejemplos guiados

Peliculas PG o G

```
SELECT title, rating
FROM film
WHERE rating IN ('PG', 'G');
```

Rentas sin devolucion

```
SELECT rental_id, rental_date, return_date
FROM rental
WHERE return_date IS NULL
LIMIT 10;
```

Ejercicios

- 2.1 Muestra peliculas con rating = 'PG'.
- 2.2 Muestra peliculas con rental_rate mayor a 2.99.
- 2.3 Lista clientes activos.
- 2.4 Busca peliculas con length entre 90 y 120.
- 2.5 Muestra peliculas con rating en ('PG', 'PG-13').
- 2.6 Encuentra actores cuyo apellido empiece con 'A'.
- 2.7 Encuentra clientes cuyo email termine en '.org'.
- 2.8 Muestra rentas que aun no tienen return_date.
- 2.9 Lista peliculas con replacement_cost >= 20 y rating = 'R'.
- 2.10 Muestra clientes de store_id 1 o store_id 2 con create_date despues de 2005-06-01.

Solucionario

2.1 Muestra peliculas con rating = 'PG'.

```
SELECT title, rating
FROM film
WHERE rating = 'PG';
```

2.2 Muestra peliculas con rental_rate mayor a 2.99.

```
SELECT title, rental_rate
FROM film
WHERE rental_rate > 2.99;
```

2.3 Lista clientes activos.

```
SELECT customer_id, first_name, last_name
FROM customer
WHERE active = 1;
```

2.4 Busca películas con length entre 90 y 120.

```
SELECT title, length
FROM film
WHERE length BETWEEN 90 AND 120;
```

2.5 Muestra películas con rating en ('PG', 'PG-13').

```
SELECT title, rating
FROM film
WHERE rating IN ('PG', 'PG-13');
```

2.6 Encuentra actores cuyo apellido empiece con 'A'.

```
SELECT first_name, last_name
FROM actor
WHERE last_name LIKE 'A%';
```

2.7 Encuentra clientes cuyo email termine en '.org'.

```
SELECT first_name, last_name, email
FROM customer
WHERE email LIKE '%.org';
```

2.8 Muestra rentas que aun no tienen return_date.

```
SELECT rental_id, customer_id, rental_date
FROM rental
WHERE return_date IS NULL;
```

2.9 Lista películas con replacement_cost >= 20 y rating = 'R'.

```
SELECT title, replacement_cost, rating
FROM film
WHERE replacement_cost >= 20 AND rating = 'R';
```

2.10 Muestra clientes de store_id 1 o store_id 2 con create_date después de 2005-06-01.

```
SELECT customer_id, store_id, create_date
FROM customer
WHERE store_id IN (1,2)
AND create_date > '2005-06-01';
```

Modulo 3. Expresiones, CASE, funciones de texto y calculos

Objetivo: Transformar columnas para construir salidas mas utiles.

Conceptos clave

- Las expresiones permiten derivar nuevas columnas sin alterar la tabla.
- CASE sirve para clasificar filas en grupos de negocio.
- Funciones de texto como CONCAT, UPPER, LOWER, LENGTH y SUBSTRING son muy usadas en limpieza.
- ROUND, ABS y operaciones aritmeticas ayudan a preparar metricas.

Ejemplos guiados

Clasifica peliculas por duracion

```
SELECT title,
       length,
       CASE
         WHEN length < 90 THEN 'Corta'
         WHEN length <= 120 THEN 'Media'
         ELSE 'Larga'
       END AS categoria_duracion
FROM film;
```

Normaliza correos

```
SELECT customer_id,
       LOWER(email) AS email_normalizado
FROM customer;
```

Ejercicios

- 3.1** Muestra title y una columna longitud_horas como length / 60 redondeado a 2 decimales.
- 3.2** Construye nombre_completo para staff.
- 3.3** Muestra title en mayusculas.
- 3.4** Cuenta la longitud del titulo de cada pelicula.
- 3.5** Extrae los primeros 3 caracteres del apellido de cada actor.
- 3.6** Clasifica clientes como 'Activo' o 'Inactivo' usando CASE.
- 3.7** Clasifica peliculas por costo de reemplazo: Bajo (<15), Medio (15-20), Alto (>20).
- 3.8** Muestra amount, amount * 1.16 como monto_con_iva y redondea a 2 decimales.
- 3.9** Usa COALESCE para mostrar original_language_id o 0 si es NULL.
- 3.10** Muestra una etiqueta de rating con texto mas descriptivo usando CASE.

Solucionario

3.1 Muestra title y una columna longitud_horas como length / 60 redondeado a 2 decimales.

```
SELECT title,
        ROUND(length / 60, 2) AS longitud_horas
FROM film;
```

3.2 Construye nombre_completo para staff.

```
SELECT CONCAT(first_name, ' ', last_name) AS nombre_completo
FROM staff;
```

3.3 Muestra title en mayusculas.

```
SELECT UPPER(title) AS titulo_mayusculas
FROM film;
```

3.4 Cuenta la longitud del titulo de cada pelicula.

```
SELECT title, LENGTH(title) AS largo_titulo
FROM film;
```

3.5 Extrae los primeros 3 caracteres del apellido de cada actor.

```
SELECT last_name, SUBSTRING(last_name, 1, 3) AS prefijo
FROM actor;
```

3.6 Clasifica clientes como 'Activo' o 'Inactivo' usando CASE.

```
SELECT customer_id,
        CASE WHEN active = 1 THEN 'Activo' ELSE 'Inactivo' END AS estado_cliente
FROM customer;
```

3.7 Clasifica peliculas por costo de reemplazo: Bajo (<15), Medio (15-20), Alto (>20).

```
SELECT title,
        replacement_cost,
        CASE
            WHEN replacement_cost < 15 THEN 'Bajo'
            WHEN replacement_cost <= 20 THEN 'Medio'
            ELSE 'Alto'
        END AS banda_costo
FROM film;
```

3.8 Muestra amount, amount * 1.16 como monto_con_iva y redondea a 2 decimales.

```
SELECT amount,
        ROUND(amount * 1.16, 2) AS monto_con_iva
FROM payment;
```

3.9 Usa COALESCE para mostrar original_language_id o 0 si es NULL.

```
SELECT film_id, COALESCE(original_language_id, 0) AS idioma_original_seguro
FROM film;
```

3.10 Muestra una etiqueta de rating con texto mas descriptivo usando CASE.

```
SELECT title,
        rating,
        CASE rating
            WHEN 'G' THEN 'Todo publico'
            WHEN 'PG' THEN 'Guia paterna'
            WHEN 'PG-13' THEN 'Mayores de 13'
            WHEN 'R' THEN 'Restringida'
            ELSE 'NC-17'
        END AS descripcion_rating
FROM film;
```

Modulo 4. Agregaciones, GROUP BY y HAVING

Objetivo: Pasar de nivel fila a nivel resumen para hacer analisis.

Conceptos clave

- COUNT cuenta filas; COUNT(columna) ignora NULL; COUNT(DISTINCT) cuenta valores unicos.
- SUM, AVG, MIN y MAX resumen valores numericos.
- GROUP BY define el grano del resultado.
- HAVING filtra despues de agrupar y se usa con agregaciones.

Ejemplos guiados

Peliculas por rating

```
SELECT rating, COUNT(*) AS total_peliculas
FROM film
GROUP BY rating
ORDER BY total_peliculas DESC;
```

Ingreso por cliente

```
SELECT customer_id, SUM(amount) AS ingreso_total
FROM payment
GROUP BY customer_id
ORDER BY ingreso_total DESC
LIMIT 10;
```

Ejercicios

- 4.1 Cuenta cuantas peliculas hay en film.
- 4.2 Cuenta clientes activos e inactivos agrupando por active.
- 4.3 Calcula AVG, MIN y MAX de replacement_cost.
- 4.4 Muestra cantidad de peliculas por rating.
- 4.5 Muestra promedio de duration por rating.
- 4.6 Cuenta cuantas rentas hizo cada cliente.
- 4.7 Suma el monto total pagado por staff.
- 4.8 Cuenta peliculas por rental_rate y ordena descendente.
- 4.9 Muestra ratings con mas de 200 peliculas usando HAVING.
- 4.10 Muestra clientes cuyo total pagado sea mayor a 150.

Solucionario

4.1 Cuenta cuantas peliculas hay en film.

```
SELECT COUNT(*) AS total_peliculas
FROM film;
```

4.2 Cuenta clientes activos e inactivos agrupando por active.

```
SELECT active, COUNT(*) AS total_clientes
FROM customer
GROUP BY active;
```

4.3 Calcula AVG, MIN y MAX de replacement_cost.

```
SELECT AVG(replacement_cost) AS promedio,
       MIN(replacement_cost) AS minimo,
       MAX(replacement_cost) AS maximo
FROM film;
```

4.4 Muestra cantidad de películas por rating.

```
SELECT rating, COUNT(*) AS total
FROM film
GROUP BY rating;
```

4.5 Muestra promedio de duration por rating.

```
SELECT rating, AVG(length) AS promedio_duracion
FROM film
GROUP BY rating;
```

4.6 Cuenta cuantas rentas hizo cada cliente.

```
SELECT customer_id, COUNT(*) AS total_rentas
FROM rental
GROUP BY customer_id
ORDER BY total_rentas DESC;
```

4.7 Suma el monto total pagado por staff.

```
SELECT staff_id, SUM(amount) AS total_cobrado
FROM payment
GROUP BY staff_id;
```

4.8 Cuenta películas por rental_rate y ordena descendente.

```
SELECT rental_rate, COUNT(*) AS total_peliculas
FROM film
GROUP BY rental_rate
ORDER BY total_peliculas DESC;
```

4.9 Muestra ratings con más de 200 películas usando HAVING.

```
SELECT rating, COUNT(*) AS total
FROM film
GROUP BY rating
HAVING COUNT(*) > 200;
```

4.10 Muestra clientes cuyo total pagado sea mayor a 150.

```
SELECT customer_id, SUM(amount) AS total_pagado
FROM payment
GROUP BY customer_id
HAVING SUM(amount) > 150
ORDER BY total_pagado DESC;
```

Modulo 5. JOINS: INNER, LEFT y modelos relacionales

Objetivo: Combinar tablas correctamente y evitar duplicados inesperados.

Conceptos clave

- INNER JOIN devuelve coincidencias en ambas tablas.
- LEFT JOIN conserva todas las filas de la tabla izquierda.
- Antes de hacer JOIN, identifica PK y FK para entender el grano.
- Sakila es ideal para practicar joins en estrella: film, category, actor, inventory, rental, payment.

Ejemplos guiados

Peliculas y categorias

```
SELECT f.title, c.name AS categoria
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id;
```

Clientes y ubicacion

```
SELECT cu.customer_id, cu.first_name, cu.last_name, a.address, ci.city, co.country
FROM customer cu
JOIN address a ON cu.address_id = a.address_id
JOIN city ci ON a.city_id = ci.city_id
JOIN country co ON ci.country_id = co.country_id;
```

Ejercicios

- 5.1 Une customer con address para mostrar direccion por cliente.
- 5.2 Une address, city y country para listar ciudad y pais.
- 5.3 Muestra title y categoria de cada pelicula.
- 5.4 Muestra actores y peliculas en las que aparecen.
- 5.5 Cuenta cuantas peliculas tiene cada categoria.
- 5.6 Lista rentas con titulo de pelicula.
- 5.7 Muestra pagos con nombre del cliente.
- 5.8 Usa LEFT JOIN para mostrar todas las peliculas y, si existe, su idioma original.
- 5.9 Muestra cada staff con su store y direccion.
- 5.10 Cuenta cuantos actores tiene cada pelicula.

Solucionario

5.1 Une customer con address para mostrar direccion por cliente.

```
SELECT c.customer_id, c.first_name, c.last_name, a.address
FROM customer c
JOIN address a ON c.address_id = a.address_id;
```

5.2 Une address, city y country para listar ciudad y pais.

```

SELECT a.address, ci.city, co.country
FROM address a
JOIN city ci ON a.city_id = ci.city_id
JOIN country co ON ci.country_id = co.country_id;

```

5.3 Muestra title y categoria de cada pelicula.

```

SELECT f.title, c.name AS categoria
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id;

```

5.4 Muestra actores y peliculas en las que aparecen.

```

SELECT a.first_name, a.last_name, f.title
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
JOIN film f ON fa.film_id = f.film_id;

```

5.5 Cuenta cuantas peliculas tiene cada categoria.

```

SELECT c.name AS categoria, COUNT(*) AS total_peliculas
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
GROUP BY c.name
ORDER BY total_peliculas DESC;

```

5.6 Lista rentas con titulo de pelicula.

```

SELECT r.rental_id, r.rental_date, f.title
FROM rental r
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id;

```

5.7 Muestra pagos con nombre del cliente.

```

SELECT p.payment_id, p.amount, c.first_name, c.last_name
FROM payment p
JOIN customer c ON p.customer_id = c.customer_id;

```

5.8 Usa LEFT JOIN para mostrar todas las peliculas y, si existe, su idioma original.

```

SELECT f.title, l.name AS idioma_original
FROM film f
LEFT JOIN language l ON f.original_language_id = l.language_id;

```

5.9 Muestra cada staff con su store y direccion.

```

SELECT s.staff_id, s.first_name, st.store_id, a.address
FROM staff s
JOIN store st ON s.store_id = st.store_id
JOIN address a ON st.address_id = a.address_id;

```

5.10 Cuenta cuantos actores tiene cada pelicula.

```

SELECT f.title, COUNT(fa.actor_id) AS total_actores
FROM film f
JOIN film_actor fa ON f.film_id = fa.film_id
GROUP BY f.title
ORDER BY total_actores DESC, f.title;

```

Modulo 6. CAST, conversiones, fechas y tiempo

Objetivo: Manipular tipos de datos y construir consultas mas realistas con tiempo.

Conceptos clave

- CAST convierte un valor a otro tipo para comparar, formatear o calcular.
- DATE(), YEAR(), MONTH(), DAY() y EXTRACT ayudan a cortar timestamps.
- DATEDIFF calcula diferencia entre fechas; TIMESTAMPDIF es aun mas flexible en algunos motores.
- En analitica es comun agrupar por dia, mes y dia de semana.

Ejemplos guiados

Pagos por dia

```
SELECT DATE(payment_date) AS fecha, SUM(amount) AS total_dia
FROM payment
GROUP BY DATE(payment_date)
ORDER BY fecha;
```

Duracion de renta

```
SELECT rental_id,
       rental_date,
       return_date,
       DATEDIFF(return_date, rental_date) AS dias_renta
FROM rental
WHERE return_date IS NOT NULL
LIMIT 10;
```

Ejercicios

- 6.1 Convierte release_year a entero usando CAST.
- 6.2 Muestra payment_date y solo la fecha.
- 6.3 Agrupa pagos por fecha.
- 6.4 Calcula el monto promedio por mes usando YEAR y MONTH.
- 6.5 Calcula dias entre rental_date y return_date para rentas devueltas.
- 6.6 Muestra clientes creados en junio de 2005.
- 6.7 Agrupa pagos por hora del dia.
- 6.8 Convierte amount a decimal con 2 posiciones usando CAST.
- 6.9 Muestra el nombre del mes de payment_date si tu motor soporta MONTHNAME.
- 6.10 Cuenta rentas por dia de la semana usando DAYNAME.

Solucionario

6.1 Convierte release_year a entero usando CAST.

```
SELECT title, CAST(release_year AS UNSIGNED) AS anio
FROM film;
```

6.2 Muestra payment_date y solo la fecha.

```
SELECT payment_date, DATE(payment_date) AS solo_fecha
FROM payment
LIMIT 10;
```

6.3 Agrupa pagos por fecha.

```
SELECT DATE(payment_date) AS fecha, COUNT(*) AS total_pagos
FROM payment
GROUP BY DATE(payment_date)
ORDER BY fecha;
```

6.4 Calcula el monto promedio por mes usando YEAR y MONTH.

```
SELECT YEAR(payment_date) AS anio,
       MONTH(payment_date) AS mes,
       AVG(amount) AS promedio_monto
FROM payment
GROUP BY YEAR(payment_date), MONTH(payment_date)
ORDER BY anio, mes;
```

6.5 Calcula días entre rental_date y return_date para rentas devueltas.

```
SELECT rental_id,
       DATEDIFF(return_date, rental_date) AS dias_renta
FROM rental
WHERE return_date IS NOT NULL;
```

6.6 Muestra clientes creados en junio de 2005.

```
SELECT customer_id, create_date
FROM customer
WHERE YEAR(create_date) = 2005 AND MONTH(create_date) = 6;
```

6.7 Agrupa pagos por hora del día.

```
SELECT HOUR(payment_date) AS hora, COUNT(*) AS total_pagos
FROM payment
GROUP BY HOUR(payment_date)
ORDER BY hora;
```

6.8 Convierte amount a decimal con 2 posiciones usando CAST.

```
SELECT amount, CAST(amount AS DECIMAL(10,2)) AS amount_decimal
FROM payment
LIMIT 10;
```

6.9 Muestra el nombre del mes de payment_date si tu motor soporta MONTHNAME.

```
SELECT payment_id, MONTHNAME(payment_date) AS nombre_mes
FROM payment
LIMIT 10;
```

6.10 Cuenta rentas por día de la semana usando DAYNAME.

```
SELECT DAYNAME(rental_date) AS dia_semana, COUNT(*) AS total_rentas
FROM rental
GROUP BY DAYNAME(rental_date);
```

Modulo 7. Subconsultas y subconsultas correlacionadas

Objetivo: Resolver preguntas donde una consulta necesita el resultado de otra.

Conceptos clave

- Las subconsultas pueden vivir en SELECT, FROM o WHERE.
- Una subconsulta escalar devuelve un solo valor, por ejemplo el promedio global.
- EXISTS es muy util para validar existencia sin traer columnas.
- Las correlacionadas se ejecutan por fila y requieren cuidado por rendimiento.

Ejemplos guiados

Peliculas por encima del promedio de duracion

```
SELECT title, length
FROM film
WHERE length > (SELECT AVG(length) FROM film);
```

Cientes por encima del promedio de gasto

```
SELECT customer_id, total_pagado
FROM (
  SELECT customer_id, SUM(amount) AS total_pagado
  FROM payment
  GROUP BY customer_id
) x
WHERE total_pagado > (
  SELECT AVG(total_cliente)
  FROM (
    SELECT SUM(amount) AS total_cliente
    FROM payment
    GROUP BY customer_id
  ) y
);
```

Ejercicios

- 7.1 Muestra peliculas cuya duracion sea mayor al promedio general.
- 7.2 Muestra peliculas cuyo replacement_cost sea igual al maximo.
- 7.3 Lista clientes que hayan hecho al menos un pago mayor a 10 usando EXISTS.
- 7.4 Muestra clientes cuyo gasto total sea superior al promedio de gasto por cliente.
- 7.5 Encuentra actores que participan en mas peliculas que el promedio de actores.
- 7.6 Muestra categorias cuya cantidad de peliculas sea igual a la maxima entre categorias.
- 7.7 Lista peliculas que nunca han sido rentadas usando NOT EXISTS.
- 7.8 Muestra clientes cuyo ultimo pago sea posterior a 2005-08-20.
- 7.9 Muestra los pagos que son mayores al pago promedio del mismo cliente (correlacionada).
- 7.10 Encuentra peliculas con mas actores que el promedio de actores por pelicula.

Solucionario

7.1 Muestra peliculas cuya duracion sea mayor al promedio general.

```

SELECT title, length
FROM film
WHERE length > (SELECT AVG(length) FROM film);

```

7.2 Muestra películas cuyo replacement_cost sea igual al máximo.

```

SELECT title, replacement_cost
FROM film
WHERE replacement_cost = (SELECT MAX(replacement_cost) FROM film);

```

7.3 Lista clientes que hayan hecho al menos un pago mayor a 10 usando EXISTS.

```

SELECT c.customer_id, c.first_name, c.last_name
FROM customer c
WHERE EXISTS (
    SELECT 1
    FROM payment p
    WHERE p.customer_id = c.customer_id
    AND p.amount > 10
);

```

7.4 Muestra clientes cuyo gasto total sea superior al promedio de gasto por cliente.

```

SELECT customer_id, SUM(amount) AS total_pagado
FROM payment
GROUP BY customer_id
HAVING SUM(amount) > (
    SELECT AVG(total_cliente)
    FROM (
        SELECT SUM(amount) AS total_cliente
        FROM payment
        GROUP BY customer_id
    ) t
);

```

7.5 Encuentra actores que participan en más películas que el promedio de actores.

```

SELECT actor_id, COUNT(*) AS total_peliculas
FROM film_actor
GROUP BY actor_id
HAVING COUNT(*) > (
    SELECT AVG(total_por_actor)
    FROM (
        SELECT COUNT(*) AS total_por_actor
        FROM film_actor
        GROUP BY actor_id
    ) x
);

```

7.6 Muestra categorías cuya cantidad de películas sea igual a la máxima entre categorías.

```

SELECT c.name, COUNT(*) AS total_peliculas
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
GROUP BY c.name
HAVING COUNT(*) = (
    SELECT MAX(total_cat)
    FROM (
        SELECT COUNT(*) AS total_cat
        FROM film_category
        GROUP BY category_id
    ) y
);

```

7.7 Lista películas que nunca han sido rentadas usando NOT EXISTS.

```
SELECT f.title
FROM film f
WHERE NOT EXISTS (
    SELECT 1
    FROM inventory i
    JOIN rental r ON i.inventory_id = r.inventory_id
    WHERE i.film_id = f.film_id
);
```

7.8 Muestra clientes cuyo ultimo pago sea posterior a 2005-08-20.

```
SELECT customer_id, MAX(payment_date) AS ultimo_pago
FROM payment
GROUP BY customer_id
HAVING MAX(payment_date) > '2005-08-20';
```

7.9 Muestra los pagos que son mayores al pago promedio del mismo cliente (correlacionada).

```
SELECT p.payment_id, p.customer_id, p.amount
FROM payment p
WHERE p.amount > (
    SELECT AVG(p2.amount)
    FROM payment p2
    WHERE p2.customer_id = p.customer_id
);
```

7.10 Encuentra peliculas con mas actores que el promedio de actores por pelicula.

```
SELECT fa.film_id, COUNT(*) AS total_actores
FROM film_actor fa
GROUP BY fa.film_id
HAVING COUNT(*) > (
    SELECT AVG(total_por_pelicula)
    FROM (
        SELECT COUNT(*) AS total_por_pelicula
        FROM film_actor
        GROUP BY film_id
    ) z
);
```

Modulo 8. CTEs y consultas por etapas

Objetivo: Escribir SQL mas legible, modular y facil de depurar.

Conceptos clave

- WITH define Common Table Expressions y ayuda a descomponer problemas.
- Una CTE no siempre mejora rendimiento, pero casi siempre mejora legibilidad.
- Es util para separar capas: base, agregacion, enriquecimiento, ranking.
- En entrevistas, una buena CTE comunica muy bien tu razonamiento.

Ejemplos guiados

Pago total por cliente con CTE

```
WITH pagos_cliente AS (  
  SELECT customer_id, SUM(amount) AS total_pagado  
  FROM payment  
  GROUP BY customer_id  
)  
SELECT *  
FROM pagos_cliente  
ORDER BY total_pagado DESC  
LIMIT 10;
```

Rentas por categoria con CTEs

```
WITH rentas_categoria AS (  
  SELECT c.name AS categoria, COUNT(*) AS total_rentas  
  FROM rental r  
  JOIN inventory i ON r.inventory_id = i.inventory_id  
  JOIN film_category fc ON i.film_id = fc.film_id  
  JOIN category c ON fc.category_id = c.category_id  
  GROUP BY c.name  
)  
SELECT *  
FROM rentas_categoria  
ORDER BY total_rentas DESC;
```

Ejercicios

- 8.1** Crea una CTE con pagos por cliente y muestra el top 10 por total_pagado.
- 8.2** Crea una CTE con total_rentas por cliente y filtra clientes con mas de 30 rentas.
- 8.3** Usa dos CTEs para obtener peliculas por categoria y luego calcular el promedio por categoria.
- 8.4** Crea una CTE diaria con ventas por fecha y luego calcula dias arriba del promedio diario.
- 8.5** Usa CTE para calcular actores por pelicula y traer solo peliculas con mas de 8 actores.
- 8.6** Construye una CTE base con pagos y otra para clasificar pagos bajos, medios y altos.
- 8.7** Usa una CTE para traer la renta mas reciente de cada cliente.
- 8.8** Crea una CTE con pagos por staff y calcula su participacion del total.
- 8.9** Genera una CTE con peliculas rentadas y otra con peliculas no rentadas usando NOT EXISTS.
- 8.10** Usa CTEs para sacar clientes, total pagado y total rentas en una sola salida.

Solucionario

8.1 Crea una CTE con pagos por cliente y muestra el top 10 por total_pagado.

```
WITH pagos_cliente AS (  
    SELECT customer_id, SUM(amount) AS total_pagado  
    FROM payment  
    GROUP BY customer_id  
)  
SELECT *  
FROM pagos_cliente  
ORDER BY total_pagado DESC  
LIMIT 10;
```

8.2 Crea una CTE con total_rentas por cliente y filtra clientes con mas de 30 rentas.

```
WITH rentas_cliente AS (  
    SELECT customer_id, COUNT(*) AS total_rentas  
    FROM rental  
    GROUP BY customer_id  
)  
SELECT *  
FROM rentas_cliente  
WHERE total_rentas > 30  
ORDER BY total_rentas DESC;
```

8.3 Usa dos CTEs para obtener peliculas por categoria y luego calcular el promedio por categoria.

```
WITH peliculas_categoria AS (  
    SELECT category_id, COUNT(*) AS total_peliculas  
    FROM film_category  
    GROUP BY category_id  
) , detalle AS (  
    SELECT c.name, pc.total_peliculas  
    FROM peliculas_categoria pc  
    JOIN category c ON pc.category_id = c.category_id  
)  
SELECT *  
FROM detalle  
ORDER BY total_peliculas DESC;
```

8.4 Crea una CTE diaria con ventas por fecha y luego calcula dias arriba del promedio diario.

```
WITH ventas_dia AS (  
    SELECT DATE(payment_date) AS fecha, SUM(amount) AS total_dia  
    FROM payment  
    GROUP BY DATE(payment_date)  
)  
SELECT *  
FROM ventas_dia  
WHERE total_dia > (SELECT AVG(total_dia) FROM ventas_dia)  
ORDER BY fecha;
```

8.5 Usa CTE para calcular actores por pelicula y traer solo peliculas con mas de 8 actores.

```
WITH actores_pelicula AS (  
    SELECT film_id, COUNT(*) AS total_actores  
    FROM film_actor  
    GROUP BY film_id  
)  
SELECT f.title, ap.total_actores  
FROM actores_pelicula ap  
JOIN film f ON ap.film_id = f.film_id  
WHERE ap.total_actores > 8  
ORDER BY ap.total_actores DESC, f.title;
```

8.6 Construye una CTE base con pagos y otra para clasificar pagos bajos, medios y altos.

```

WITH base AS (
  SELECT payment_id, customer_id, amount
  FROM payment
), clasificada AS (
  SELECT *,
    CASE
      WHEN amount < 3 THEN 'Bajo'
      WHEN amount <= 7 THEN 'Medio'
      ELSE 'Alto'
    END AS banda
  FROM base
)
SELECT banda, COUNT(*) AS total
FROM clasificada
GROUP BY banda;

```

8.7 Usa una CTE para traer la renta mas reciente de cada cliente.

```

WITH ultima_renta AS (
  SELECT customer_id, MAX(rental_date) AS fecha_ultima_renta
  FROM rental
  GROUP BY customer_id
)
SELECT *
FROM ultima_renta
ORDER BY fecha_ultima_renta DESC;

```

8.8 Crea una CTE con pagos por staff y calcula su participacion del total.

```

WITH pagos_staff AS (
  SELECT staff_id, SUM(amount) AS total_staff
  FROM payment
  GROUP BY staff_id
)
SELECT staff_id,
  total_staff,
  ROUND(100 * total_staff / (SELECT SUM(total_staff) FROM pagos_staff), 2) AS pct_total
FROM pagos_staff;

```

8.9 Genera una CTE con peliculas rentadas y otra con peliculas no rentadas usando NOT EXISTS.

```

WITH peliculas_rentadas AS (
  SELECT DISTINCT i.film_id
  FROM inventory i
  JOIN rental r ON i.inventory_id = r.inventory_id
)
SELECT f.title
FROM film f
WHERE NOT EXISTS (
  SELECT 1
  FROM peliculas_rentadas pr
  WHERE pr.film_id = f.film_id
);

```

8.10 Usa CTEs para sacar clientes, total pagado y total rentas en una sola salida.

```
WITH pagos AS (  
  SELECT customer_id, SUM(amount) AS total_pagado  
  FROM payment  
  GROUP BY customer_id  
) , rentas AS (  
  SELECT customer_id, COUNT(*) AS total_rentas  
  FROM rental  
  GROUP BY customer_id  
)  
SELECT c.customer_id, c.first_name, c.last_name,  
       p.total_pagado, r.total_rentas  
FROM customer c  
LEFT JOIN pagos p ON c.customer_id = p.customer_id  
LEFT JOIN rentas r ON c.customer_id = r.customer_id;
```

Modulo 9. Funciones de ventana

Objetivo: Analizar sin perder el detalle de fila.

Conceptos clave

- Las ventanas calculan metricas sobre un conjunto relacionado de filas sin colapsarlo como GROUP BY.
- OVER() define la ventana; PARTITION BY divide grupos y ORDER BY marca secuencia.
- ROW_NUMBER, RANK y DENSE_RANK sirven para rankings.
- SUM, AVG y LAG/LEAD son excelentes para acumulados y comparaciones temporales.

Ejemplos guiados

Ranking de clientes por gasto

```
WITH pagos_cliente AS (  
  SELECT customer_id, SUM(amount) AS total_pagado  
  FROM payment  
  GROUP BY customer_id  
)  
SELECT customer_id, total_pagado,  
       DENSE_RANK() OVER (ORDER BY total_pagado DESC) AS ranking  
FROM pagos_cliente;
```

Acumulado diario

```
WITH ventas_dia AS (  
  SELECT DATE(payment_date) AS fecha, SUM(amount) AS total_dia  
  FROM payment  
  GROUP BY DATE(payment_date)  
)  
SELECT fecha, total_dia,  
       SUM(total_dia) OVER (ORDER BY fecha) AS acumulado  
FROM ventas_dia;
```

Ejercicios

- 9.1 Calcula ROW_NUMBER sobre peliculas ordenadas por length desc.
- 9.2 Calcula RANK y DENSE_RANK sobre replacement_cost desc.
- 9.3 Haz ranking de clientes por total_pagado.
- 9.4 Para cada rating, numera peliculas por duracion descendente.
- 9.5 Calcula venta acumulada por fecha.
- 9.6 Calcula promedio movil de 3 dias sobre total_dia.
- 9.7 Usa LAG para comparar el total del dia actual vs el anterior.
- 9.8 Obtiene el top 3 de clientes por gasto usando una subconsulta con ROW_NUMBER.
- 9.9 Para cada categoria, rankea peliculas por cantidad de rentas.
- 9.10 Divide clientes en cuartiles de gasto usando NTILE(4).

Solucionario

9.1 Calcula ROW_NUMBER sobre peliculas ordenadas por length desc.

```
SELECT title, length,
       ROW_NUMBER() OVER (ORDER BY length DESC) AS fila
FROM film;
```

9.2 Calcula RANK y DENSE_RANK sobre replacement_cost desc.

```
SELECT title, replacement_cost,
       RANK() OVER (ORDER BY replacement_cost DESC) AS rk,
       DENSE_RANK() OVER (ORDER BY replacement_cost DESC) AS dense_rk
FROM film;
```

9.3 Haz ranking de clientes por total_pagado.

```
WITH pagos_cliente AS (
  SELECT customer_id, SUM(amount) AS total_pagado
  FROM payment
  GROUP BY customer_id
)
SELECT customer_id, total_pagado,
       DENSE_RANK() OVER (ORDER BY total_pagado DESC) AS ranking
FROM pagos_cliente;
```

9.4 Para cada rating, numera películas por duración descendente.

```
SELECT title, rating, length,
       ROW_NUMBER() OVER (PARTITION BY rating ORDER BY length DESC) AS rn_rating
FROM film;
```

9.5 Calcula venta acumulada por fecha.

```
WITH ventas_dia AS (
  SELECT DATE(payment_date) AS fecha, SUM(amount) AS total_dia
  FROM payment
  GROUP BY DATE(payment_date)
)
SELECT fecha, total_dia,
       SUM(total_dia) OVER (ORDER BY fecha) AS venta_acumulada
FROM ventas_dia;
```

9.6 Calcula promedio móvil de 3 días sobre total_dia.

```
WITH ventas_dia AS (
  SELECT DATE(payment_date) AS fecha, SUM(amount) AS total_dia
  FROM payment
  GROUP BY DATE(payment_date)
)
SELECT fecha, total_dia,
       AVG(total_dia) OVER (ORDER BY fecha ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS promedio_movil_3
FROM ventas_dia;
```

9.7 Usa LAG para comparar el total del día actual vs el anterior.

```
WITH ventas_dia AS (
  SELECT DATE(payment_date) AS fecha, SUM(amount) AS total_dia
  FROM payment
  GROUP BY DATE(payment_date)
)
SELECT fecha, total_dia,
       LAG(total_dia) OVER (ORDER BY fecha) AS total_dia_anterior,
       total_dia - LAG(total_dia) OVER (ORDER BY fecha) AS variacion
FROM ventas_dia;
```

9.8 Obtiene el top 3 de clientes por gasto usando una subconsulta con ROW_NUMBER.

```

WITH pagos_cliente AS (
  SELECT customer_id, SUM(amount) AS total_pagado
  FROM payment
  GROUP BY customer_id
), ranking AS (
  SELECT customer_id, total_pagado,
         ROW_NUMBER() OVER (ORDER BY total_pagado DESC) AS rn
  FROM pagos_cliente
)
SELECT *
FROM ranking
WHERE rn <= 3;

```

9.9 Para cada categoria, rankea películas por cantidad de rentas.

```

WITH rentas_categoria AS (
  SELECT c.name AS categoria, f.title, COUNT(*) AS total_rentas
  FROM rental r
  JOIN inventory i ON r.inventory_id = i.inventory_id
  JOIN film f ON i.film_id = f.film_id
  JOIN film_category fc ON f.film_id = fc.film_id
  JOIN category c ON fc.category_id = c.category_id
  GROUP BY c.name, f.title
)
SELECT categoria, title, total_rentas,
       DENSE_RANK() OVER (PARTITION BY categoria ORDER BY total_rentas DESC) AS rk
FROM rentas_categoria;

```

9.10 Divide clientes en cuartiles de gasto usando NTILE(4).

```

WITH pagos_cliente AS (
  SELECT customer_id, SUM(amount) AS total_pagado
  FROM payment
  GROUP BY customer_id
)
SELECT customer_id, total_pagado,
       NTILE(4) OVER (ORDER BY total_pagado DESC) AS cuartil_gasto
FROM pagos_cliente;

```

Modulo 10. Analisis reales con Sakila

Objetivo: Resolver casos mas cercanos a trabajo o entrevista.

Conceptos clave

- Primero define el grano: cliente, pelicula, dia, categoria o tienda.
- Valida si una metrica es transaccional, inventario o maestra.
- Entrega salidas interpretables: ranking, porcentaje, participacion, tendencia.
- Apoya el analisis con nombres claros y filtros explicitos.

Ejemplos guiados

Top categorias por ingreso

```
SELECT c.name AS categoria, SUM(p.amount) AS ingreso_total
FROM payment p
JOIN rental r ON p.rental_id = r.rental_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film_category fc ON i.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
GROUP BY c.name
ORDER BY ingreso_total DESC;
```

Clientes de alto valor

```
WITH pagos_cliente AS (
  SELECT customer_id, SUM(amount) AS total_pagado
  FROM payment
  GROUP BY customer_id
)
SELECT customer_id, total_pagado
FROM pagos_cliente
WHERE total_pagado >= 180
ORDER BY total_pagado DESC;
```

Ejercicios

- 10.1** Calcula ingreso total por categoria.
- 10.2** Calcula top 10 clientes por gasto total con nombre completo.
- 10.3** Saca ticket promedio por cliente.
- 10.4** Mide cuantas peliculas distintas ha rentado cada cliente.
- 10.5** Calcula la participacion porcentual de cada rating sobre el total de peliculas.
- 10.6** Identifica clientes sin pagos usando LEFT JOIN.
- 10.7** Calcula ingreso por store.
- 10.8** Obtiene top 5 peliculas mas rentadas.
- 10.9** Calcula dias con mayor ingreso total.
- 10.10** Construye un ranking por categoria y pelicula con las 3 peliculas mas rentadas por categoria.

Solucionario

10.1 Calcula ingreso total por categoria.

```

SELECT c.name AS categoria, SUM(p.amount) AS ingreso_total
FROM payment p
JOIN rental r ON p.rental_id = r.rental_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film_category fc ON i.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
GROUP BY c.name
ORDER BY ingreso_total DESC;

```

10.2 Calcula top 10 clientes por gasto total con nombre completo.

```

SELECT c.customer_id,
       CONCAT(c.first_name, ' ', c.last_name) AS cliente,
       SUM(p.amount) AS total_pagado
FROM customer c
JOIN payment p ON c.customer_id = p.customer_id
GROUP BY c.customer_id, cliente
ORDER BY total_pagado DESC
LIMIT 10;

```

10.3 Saca ticket promedio por cliente.

```

SELECT customer_id, AVG(amount) AS ticket_promedio
FROM payment
GROUP BY customer_id
ORDER BY ticket_promedio DESC;

```

10.4 Mide cuantas peliculas distintas ha rentado cada cliente.

```

SELECT r.customer_id, COUNT(DISTINCT i.film_id) AS peliculas_distintas
FROM rental r
JOIN inventory i ON r.inventory_id = i.inventory_id
GROUP BY r.customer_id
ORDER BY peliculas_distintas DESC;

```

10.5 Calcula la participacion porcentual de cada rating sobre el total de peliculas.

```

SELECT rating,
       COUNT(*) AS total_peliculas,
       ROUND(100 * COUNT(*) / (SELECT COUNT(*) FROM film), 2) AS pct_total
FROM film
GROUP BY rating
ORDER BY pct_total DESC;

```

10.6 Identifica clientes sin pagos usando LEFT JOIN.

```

SELECT c.customer_id, c.first_name, c.last_name
FROM customer c
LEFT JOIN payment p ON c.customer_id = p.customer_id
WHERE p.payment_id IS NULL;

```

10.7 Calcula ingreso por store.

```

SELECT s.store_id, SUM(p.amount) AS ingreso_total
FROM payment p
JOIN staff st ON p.staff_id = st.staff_id
JOIN store s ON st.store_id = s.store_id
GROUP BY s.store_id;

```

10.8 Obtiene top 5 peliculas mas rentadas.

```
SELECT f.title, COUNT(*) AS total_rentas
FROM rental r
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
GROUP BY f.title
ORDER BY total_rentas DESC, f.title
LIMIT 5;
```

10.9 Calcula días con mayor ingreso total.

```
SELECT DATE(payment_date) AS fecha, SUM(amount) AS ingreso_total
FROM payment
GROUP BY DATE(payment_date)
ORDER BY ingreso_total DESC
LIMIT 10;
```

10.10 Construye un ranking por categoría y película con las 3 películas más rentadas por categoría.

```
WITH base AS (
  SELECT c.name AS categoria, f.title, COUNT(*) AS total_rentas
  FROM rental r
  JOIN inventory i ON r.inventory_id = i.inventory_id
  JOIN film f ON i.film_id = f.film_id
  JOIN film_category fc ON f.film_id = fc.film_id
  JOIN category c ON fc.category_id = c.category_id
  GROUP BY c.name, f.title
), ranking AS (
  SELECT *, DENSE_RANK() OVER (PARTITION BY categoria ORDER BY total_rentas DESC) AS rk
  FROM base
)
SELECT *
FROM ranking
WHERE rk <= 3
ORDER BY categoria, rk, title;
```

Modulo 11. Calidad, depuracion y patrones profesionales

Objetivo: Escribir SQL que sea correcto, mantenible y facil de revisar.

Conceptos clave

- Siempre revisa el grano antes de agregar: payment no tiene el mismo grano que customer.
- Si una metrica se duplica despues de un JOIN, probablemente cambiaste el grano sin darte cuenta.
- Usa nombres claros, CTEs y comentarios fuera de la celda SQL cuando practiques.
- Valida con consultas de control: COUNT(*), COUNT(DISTINCT), MIN/MAX, filas top y anti-joins.

Ejemplos guiados

Control de duplicados

```
SELECT COUNT(*) AS filas,  
       COUNT(DISTINCT customer_id) AS clientes_unicos  
FROM customer;
```

Anti-join para revisar peliculas no rentadas

```
SELECT f.film_id, f.title  
FROM film f  
LEFT JOIN inventory i ON f.film_id = i.film_id  
LEFT JOIN rental r ON i.inventory_id = r.inventory_id  
WHERE r.rental_id IS NULL;
```

Ejercicios

- 11.1 Cuenta filas y clientes unicos en customer.
- 11.2 Valida si payment tiene payment_id unico.
- 11.3 Compara COUNT(*) vs COUNT(return_date) en rental.
- 11.4 Haz una consulta de control para ver el minimo y maximo de amount en payment.
- 11.5 Valida cuantas peliculas distintas hay en inventory.
- 11.6 Encuentra posibles duplicados de email en customer.
- 11.7 Usa LEFT JOIN para revisar clientes sin rentas.
- 11.8 Compara el total de payment vs el total luego de unir customer sin agrupar de mas.
- 11.9 Mide el total de payment por rating, cuidando el join a rental -> inventory -> film.
- 11.10 Escribe una consulta de validacion para revisar si hay peliculas sin categoria.

Solucionario

11.1 Cuenta filas y clientes unicos en customer.

```
SELECT COUNT(*) AS filas, COUNT(DISTINCT customer_id) AS clientes_unicos  
FROM customer;
```

11.2 Valida si payment tiene payment_id unico.

```
SELECT COUNT(*) AS filas, COUNT(DISTINCT payment_id) AS ids_unicos
FROM payment;
```

11.3 Compara COUNT(*) vs COUNT(return_date) en rental.

```
SELECT COUNT(*) AS filas_totales, COUNT(return_date) AS rentas_devueltas
FROM rental;
```

11.4 Haz una consulta de control para ver el minimo y maximo de amount en payment.

```
SELECT MIN(amount) AS monto_minimo, MAX(amount) AS monto_maximo
FROM payment;
```

11.5 Valida cuantas peliculas distintas hay en inventory.

```
SELECT COUNT(DISTINCT film_id) AS peliculas_en_inventario
FROM inventory;
```

11.6 Encuentra posibles duplicados de email en customer.

```
SELECT email, COUNT(*) AS total
FROM customer
GROUP BY email
HAVING COUNT(*) > 1;
```

11.7 Usa LEFT JOIN para revisar clientes sin rentas.

```
SELECT c.customer_id, c.first_name, c.last_name
FROM customer c
LEFT JOIN rental r ON c.customer_id = r.customer_id
WHERE r.rental_id IS NULL;
```

11.8 Compara el total de payment vs el total luego de unir customer sin agrupar de mas.

```
SELECT SUM(p.amount) AS total_payment
FROM payment p;
```

11.9 Mide el total de payment por rating, cuidando el join a rental -> inventory -> film.

```
SELECT f.rating, SUM(p.amount) AS total_payment
FROM payment p
JOIN rental r ON p.rental_id = r.rental_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
GROUP BY f.rating;
```

11.10 Escribe una consulta de validacion para revisar si hay peliculas sin categoria.

```
SELECT f.film_id, f.title
FROM film f
LEFT JOIN film_category fc ON f.film_id = fc.film_id
WHERE fc.category_id IS NULL;
```

Consejos finales para volverte pro

- No memorices solo sintaxis: entiende el grano de cada tabla y la logica del negocio.
- Cada vez que hagas un JOIN, pregunta: que fila representa mi salida ahora?
- Siempre valida con consultas de control antes de presentar un numero.
- Practica escribiendo primero tu SQL en capas: base -> agregacion -> ranking -> presentacion.
- Convierte tus consultas en historias: que pregunta responde, que metrica construye y como validaste el resultado.

Siguiente paso recomendado: convertir varios modulos en notebooks separados y resolverlos sin ver el solucionario. Despues, compara rendimiento y estilo entre tus consultas y las propuestas aqui.